

Vendor Invoice Intelligence & Payment Monitoring System

1. Business Scenario / Objective

Vendors depend on timely invoice payments to manage their cash flow and daily operations. Delayed payments, incomplete invoice details, duplicate submissions, and partial payments can make it difficult to determine how much is still owed.

Organizations paying these vendors face similar challenges when tracking invoices across spreadsheets, different currencies, and multiple payment transactions.

Develop a Python desktop application that helps an organization manage vendor invoices, validate records, track payments, identify overdue balances, and generate vendor-wise summaries. The system uses Tkinter for the interface, SQLite for local storage, and published exchange rates for INR reporting.

2. Challenges Addressed

Challenge	How the application helps
Delayed payments affect vendor cash flow	Displays overdue balances, due dates, and alerts
Partial payments create uncertainty about remaining dues	Stores separate payments and calculates outstanding balances
Duplicate invoices can cause repeated processing	Checks invoice IDs and supplier/invoice-number combinations
Missing or inconsistent details delay processing	Validates required fields, dates, amounts, and supplier details
Different currencies make consolidated reporting difficult	Preserves original amounts and calculates INR equivalents
Spreadsheet records are difficult to search and compare	Provides searchable tables, status filters, and column sorting
Incorrect entries require traceable correction	Supports confirmed deletion and replacement with audit records
Manual summaries are time-consuming	Generates invoice reports and vendor-wise summaries

3. Input Data / Files

The application accepts:

- Manual entry through an invoice form.
- CSV files containing invoice records.
- Excel .xlsx files, using the first worksheet.
- Online or manually entered exchange rates with effective dates and sources.

Main invoice fields include:

- Invoice ID and invoice number
- Supplier ID, name, and optional tax ID
- Invoice date, submission date, and due date
- Invoice amount and currency
- Payment amount and payment date
- Payment method and reference
- Active or cancelled state

The supplied synthetic dataset contains 200 invoice records, covering paid, unpaid, partially paid, overdue, and cancelled invoices.

Scope assumptions:

- Each invoice has one original currency.
- Payments are recorded in that invoice's currency.
- Multiple payments can be recorded against an invoice.
- Supported currencies are INR, USD, EUR, GBP, CNY, SEK, BRL, CAD, and JPY.
- Fractional amounts are supported up to 12 decimal places.
- Subtotal and tax verification are outside the implemented scope because the selected dataset does not contain those fields.

4. Functional Requirements

Invoice entry and validation

- Provide a clearly labelled form for manual invoice entry.
- Import invoice records from CSV and Excel.
- Validate required fields, dates, numeric amounts, and payment consistency.
- Reject duplicate invoice IDs and repeated supplier/invoice-number combinations.
- Report invalid imported rows without preventing valid rows from being saved.

Local storage and OOP design

- Store suppliers, invoices, payments, exchange rates, and audit records in SQLite.
- Organize responsibilities using classes such as Invoice, InvoiceRepository, InvoiceService, CurrencyConverter, and ReportGenerator.
- Use reusable validation and calculation functions across manual entry and imports.

Payment tracking

- Record individual payments against invoices.
- Calculate total paid and outstanding balances.
- Display unpaid, partially paid, paid, and cancelled statuses.
- Prevent overpayments and invalid payment entries.
- Distinguish currently overdue invoices from invoices that were fully paid late.

Overdue alerts

- Identify outstanding invoices after their due dates.
- Show overdue amounts grouped by currency.
- Display warnings when overdue details change.
- Check periodically while the application remains open.

Currency conversion

- Load dated default reference rates and request newer published rates online.
- Retain saved rates when online retrieval is unavailable.
- Allow manual exchange-rate updates with a date and source.
- Preserve original invoice amounts and rate history.
- Calculate INR equivalents using rates applicable to the selected reporting date.

Friendly interface and reporting

- Provide tabs for Overview, Add invoice, Invoices & payments, Exchange rates, and Import & export.
- Support searching, payment-status filtering, and ascending/descending column sorting.
- Generate invoice reports, import exception reports, and vendor summaries.
- Export reports as CSV files.

Corrections and deletion

- Keep saved invoice and payment records read-only.
- Correct records through confirmed deletion and replacement.
- Preserve audit snapshots for individual deletions.
- Provide a database-clear option requiring the exact password "Delete".

5. Main Calculations

Calculation	Formula or rule
Total paid	Sum of recorded payments through the reporting date
Outstanding balance	Invoice amount – total paid
Partially paid	Paid amount is greater than zero but less than invoice amount
Overdue	Positive outstanding balance after the due date; excludes cancelled invoices
Days overdue	Reporting date – due date, with a minimum of zero
INR equivalent	Original outstanding amount × applicable INR exchange rate
Vendor summary	Totals grouped by supplier and original currency

Calculations use Python's **Decimal** to preserve monetary precision. INR conversion results are rounded to two decimal places.

6. Team Allocation

Participant 1	Participant 2
Define invoice and payment data models	Develop the Tkinter interface
Implement SQLite storage	Implement exchange-rate handling
Implement validation and duplicate checks	Implement reports and vendor summaries
Implement invoice/payment operations and imports	Implement alerts, filters, and sorting
Test validation, storage, and payment calculations	Test conversion, reporting, and interface helpers

Both members handle integration, code review, and documentation.

7. Course Skills Applied

Python fundamentals, OOP and encapsulation, functions, modules, lists, dictionaries, sets, tuples, file handling, CSV processing, Excel handling with openpyxl, SQLite, Decimal, date/time operations, exceptions, logging, Tkinter, HTTP requests, threading, queues, pytest, and AI-assisted development.

8. Expected Deliverables / Output

- Python-only desktop application with organized modules
- Local SQLite database
- Sample invoice dataset
- Invoice report with balances and payment statuses
- Vendor-wise financial summary
- Import exception report
- Application log and database audit records
- `requirement.txt` with dependencies
- Detailed README with installation steps and flow diagrams
- Word project report
- Pytest suite—the latest functional run passed 40 tests
- Evidence of AI-assisted development and verification